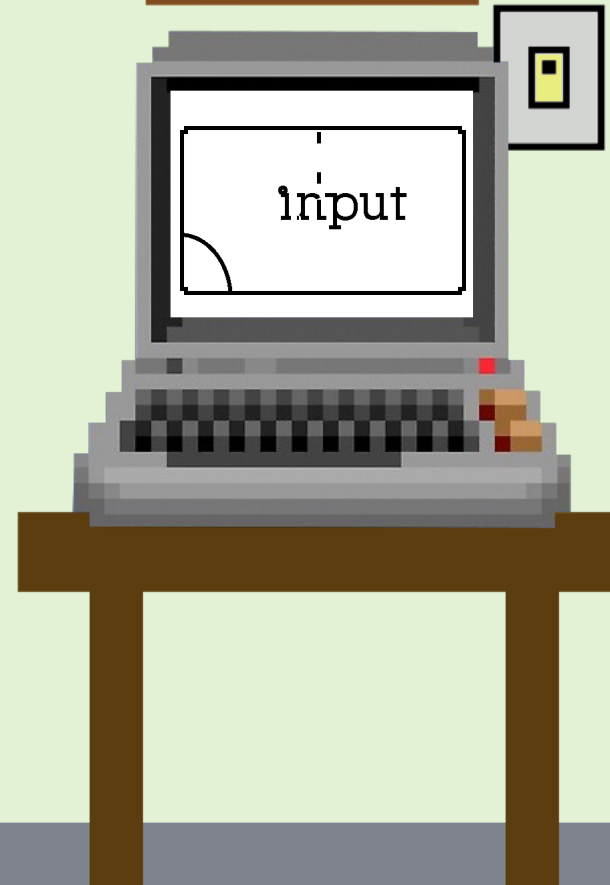


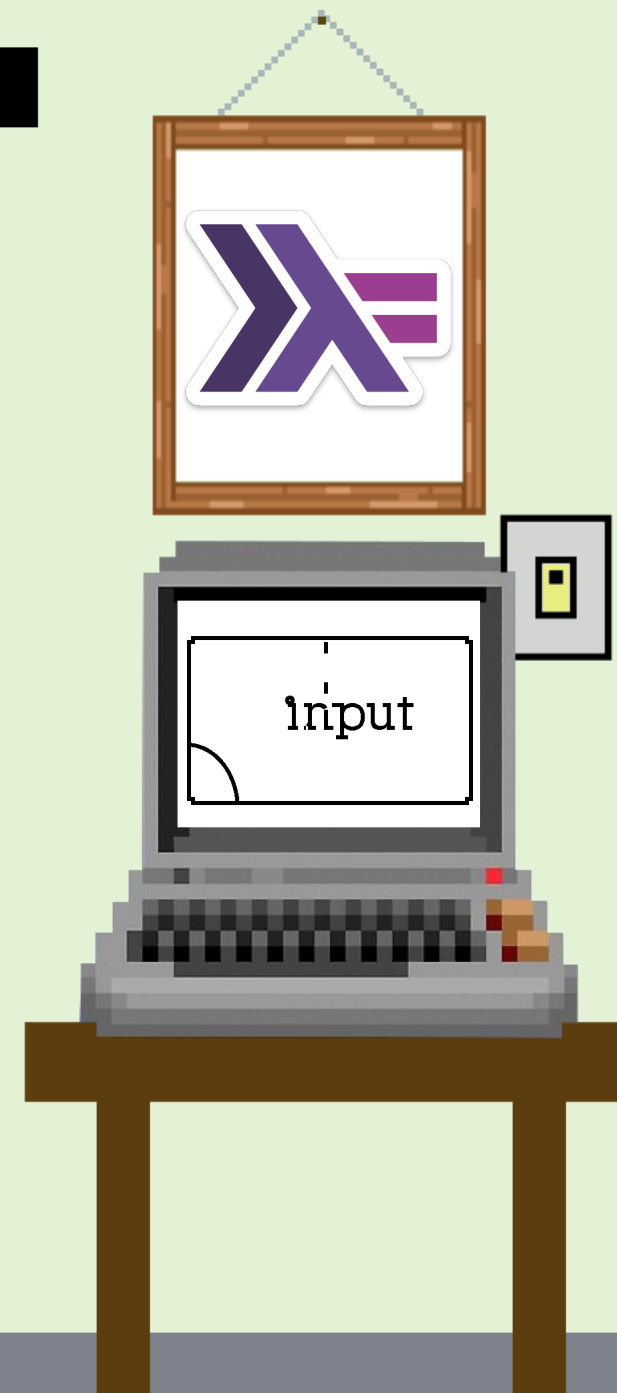
Entrada e Saída - IO

Programação Funcional



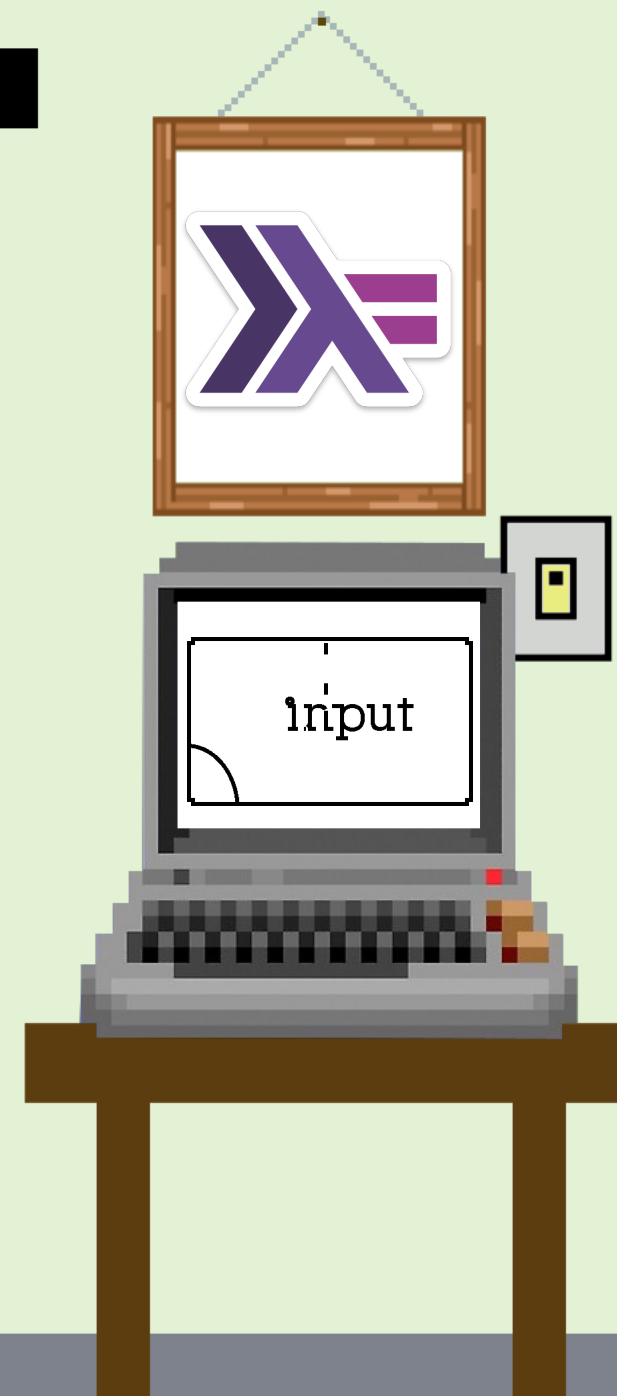
Entrada e Saída - IO

- Linguagens Funcionais puras
 - Resultado de uma função depende unicamente dos seus argumentos
 - Não existem efeitos colaterais
- Dúvidas
 - `x = getchar ()`
 - `y = getchar ()`
- Entrada e saída representa uma ação com o mundo
 - Executada pelo sistema para interagir com o mundo e retornar um valor obtido através desta interação
- Tipo `IO` representa essas ações
 - Retornam um valor do tipo `a` (`IO a` é um tipo abstrato)



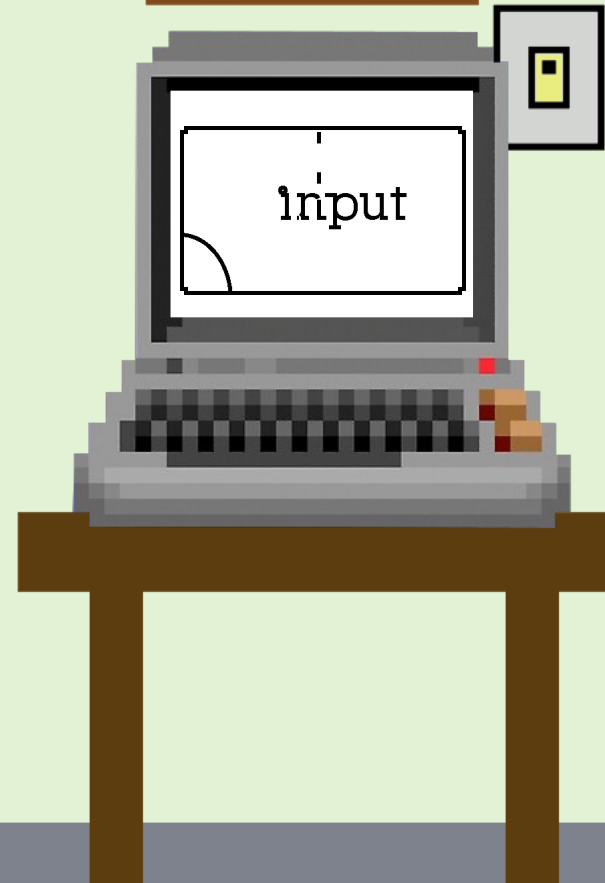
Entrada e Saída - IO

- Necessidade de representar o tipo de uma expressão que possui um valor irrelevante
- Tipos das ações de entrada e saída
 - Não possuem valor interessante para o retorno
 - O importante é o efeito da ação no mundo
- Tipo cujo conjunto de valores é vazio ou unitário
 - Não há nenhum valor
 - Há exatamente um valor desse tipo
- Haskell usa o tipo `unit ()`
 - Chamado de tupla vazia



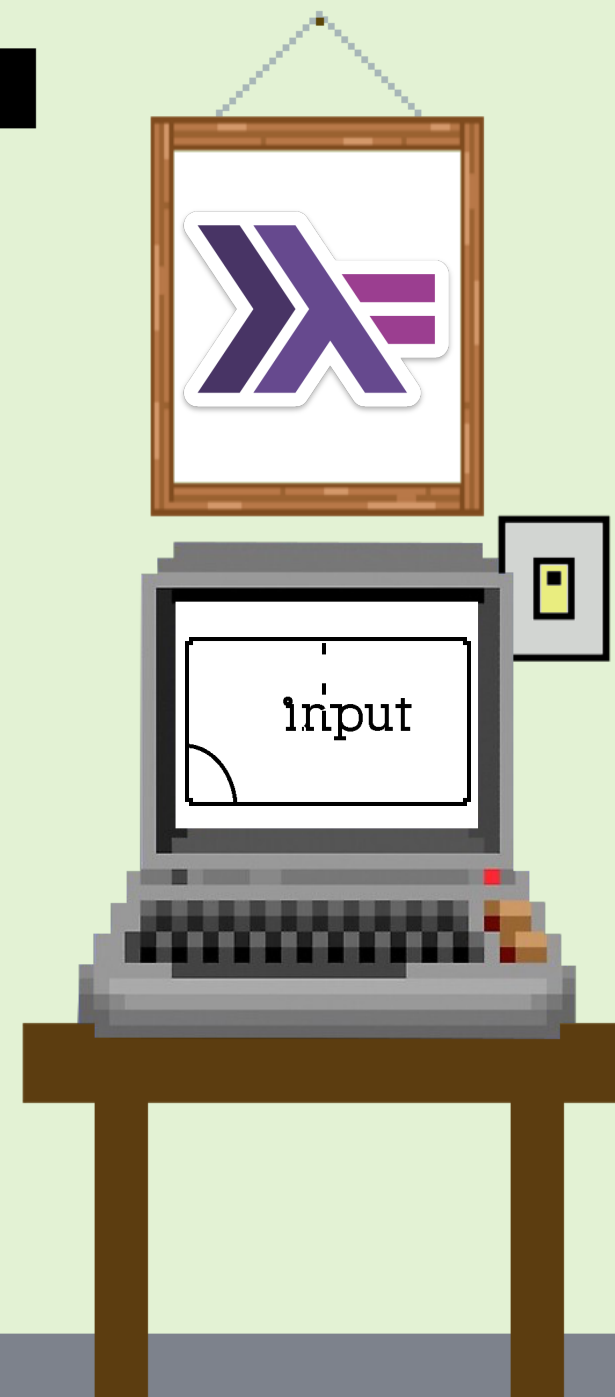
Saída Padrão

- `putChar :: Char -> IO ()`
- `putStr :: String -> IO ()`
- `putStrLn :: String -> IO ()`
- `print :: Show a => a -> IO ()`
 - `show :: Show a => a -> String`



Entrada Padrão

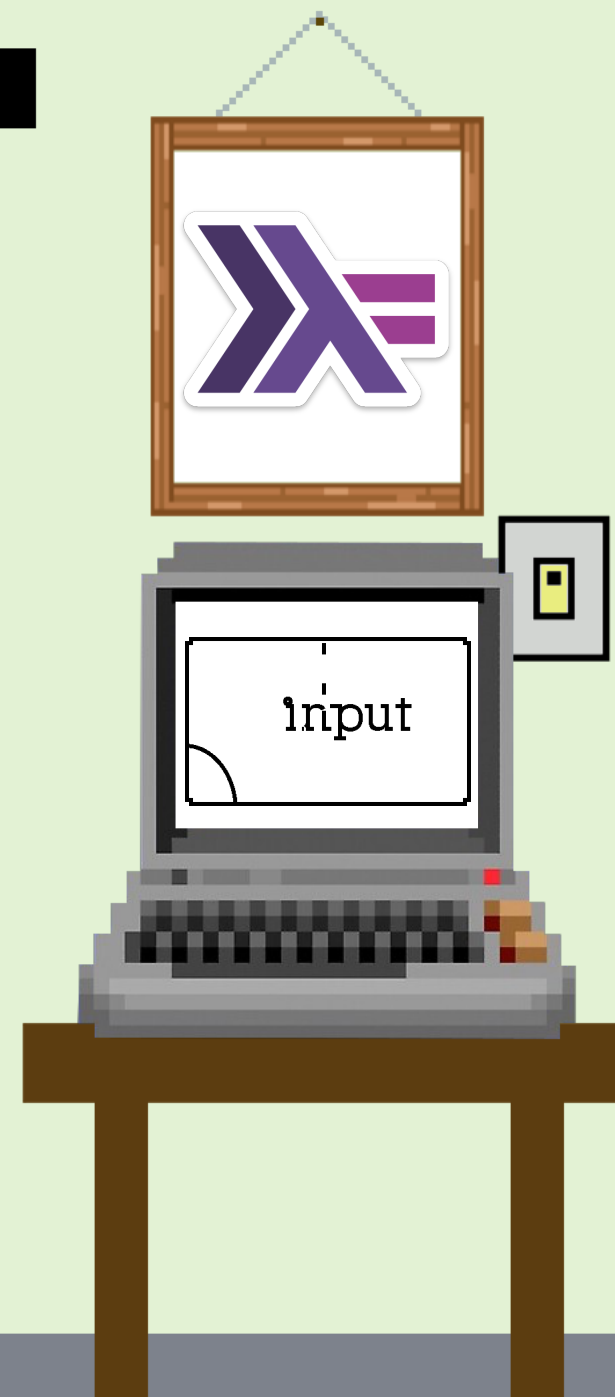
- `getChar :: IO Char`
- `getLine :: IO String`
- `getContents :: IO String`
- `readLn :: Read a => IO a`
 - Retorno é um tipo convertido a partir de `String`



E/S Haskell

- Programas são uma coleção de módulos
- Um dos módulos deve ser chamado Main
 - Exportar a variável main (tipo `IO t`)
 - O seu resultado é descartado
- Exemplo

```
main :: IO ()  
main = putStr "IFMA"
```



Combinando Ações

- Expressão **do**
 - Permite combinar ações executadas em sequência
- `do { ação 1 ; ...; ação n ; expressao }`
 - Ações (`padrao <- expressao`)
 - Declarações (**let**)
- Exemplo

```
main :: IO ()
main = do
  putChar 'I'
  putChar 'F'
  putChar 'm'
  putChar 'a'
```



Exemplos

- Ler e imprimir caractere

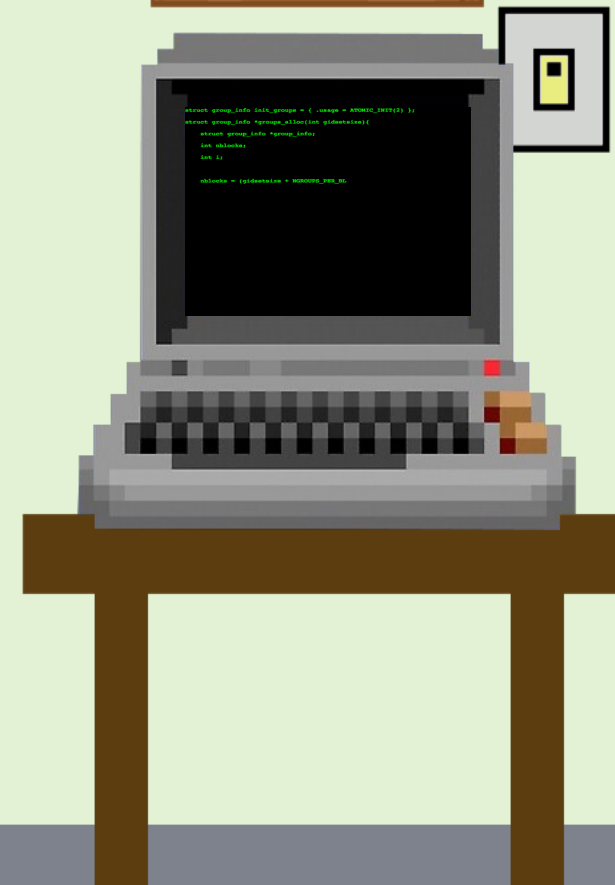
```
main :: IO ()  
main = do  
  character <- getChar  
  putChar character
```



Exemplos

- Ler e somar números

```
main :: IO ()
main = do
  putStrLn "Digite um numero:"
  s1 <- getLine
  putStrLn "Digite outro numero:"
  s2 <- getLine
  putStr "Soma dos numeros digitados: "
  let n1 = read s1 :: Double
  let n2 = read s2 :: Double
  putStrLn (show (n1 + n2))
```



Buffers de Saída

- O dispositivo padrão de saída possui um Buffer
 - Área da memória que armazena os caracteres a serem enviados para o dispositivo de saída
 - Geralmente transferida para o dispositivo de saída quando o buffer está cheio
 - Melhor desempenho
 - Podem gerar saídas não esperadas
- Sincronização
 - Função `hFlush`
 - Função `hSetBuffering`



Exemplos

- Ler e somar números

```
import System.IO
```

```
main :: IO ()
```

```
main = do
```

```
    hSetBuffering stdout NoBuffering
```

```
    putStrLn "Digite um numero:"
```

```
    s1 <- getLine
```

```
    putStrLn "Digite outro numero:"
```

```
    s2 <- getLine
```

```
    putStr "Soma dos numeros digitados: "
```

```
    putStrLn (show (read s1 + read s2))
```

